# C PROGRAMMING — ASSIGNMENT 1

## Task 4: Use Macros Safely

Marks: 4 | Total assignment: 50 marks

### Safe macros

Macros are expanded by the preprocessor before compilation. Always parenthesize each parameter and the entire expression. #define SQUARE(x) ((x) * (x)) correctly handles SQUARE(a+1), whereas x*x expands without grouping and can change operator precedence. Even a parenthesized macro may evaluate an argument twice, so SQUARE(i++) is unsafe. MAX(a,b) may also evaluate one argument more than once; avoid side effects in macro arguments. A function is usually safer.

### Preprocessor and complexity

Run gcc -E task4.c to inspect expanded source. A macro has no call overhead as a textual substitution, but its code is emitted at each use and may increase code size with k uses. A static inline function has typed arguments and only one source definition; the compiler may inline it or emit a call. Runtime cost depends on optimization and compiler decisions, so measure rather than assume.

### C Source Code / Implementation

```
#include <stdio.h>
#define RUPEES_TO_PAISE(r) ((long long)(r) * 100LL)
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) ((a) > (b) ? (a) : (b))

static inline long long square_ll(long long x) { return x * x; }
static inline int max_int(int a, int b) { return a > b ? a : b; }

int main(void) {
    int a = 3, b = 7;
    printf("SQUARE(a+1)=%d\n", SQUARE(a + 1));
    printf("MAX(a,b)=%d\n", MAX(a, b));
    printf("Rs 12 to paise=%lld\n", RUPEES_TO_PAISE(12));
    printf("inline square=%lld, inline max=%d\n",
           square_ll(4), max_int(a, b));
    /* Do not write SQUARE(a++) or MAX(a++, b): arguments can be evaluated repeatedly. */
    return 0;
}
```

### Submission note

Compile and run the code on your own system, save actual terminal output/screenshots, and include them in your report. Review the code and be prepared to explain it during the viva. Any machine-specific measurements must come from your own execution.